



Department of Computer Science, Wenzhou-Kean University

Efficient task scheduling is essential for modern computing systems, especially when workloads vary in size, arrival patterns, and urgency. Traditional algorithms, such as FCFS, SJF, RR, and EDF, perform well under specific conditions but are difficult to apply universally.

This project includes local simulation experiments and cloud monitoring on AWS EC2 using Docker, Prometheus, and Grafana. By conducting system tests on batch, mixed, and peak workloads, traditional algorithms are compared with hybrid approaches to determine performance trade-offs and optimal scheduling strategies.

```
graph TD; A[Workload Input  
(experiments/workloads/*.json)] --> B[Experiment Runner  
(runner.py)]; B --> C[Raw Results  
(CSV per experiment)]; C --> D[Result Visualization  
(plot_results.py)]; D --> E[Final Output: Algorithm  
Comparison Plots  
(PNG graphs per workload &  
strategy)]; F[AWS EC2 Instance  
(upload code + docker)] --> G[Build Docker image  
Run hybrid-api (8080)]; G --> H[Prometheus Container  
Scrape / metrics]; H --> I[Grafana Visualization  
Dashboard (P50/P90...)]
```

The flowchart illustrates the workflow for running experiments on AWS EC2 instances. It is organized into two parallel vertical flows.

Left Flow (Local/Script-based):

- Workload Input** (experiments/workloads/*.json) - Green box
- Experiment Runner** (runner.py) - Blue box
- Raw Results** (CSV per experiment) - Blue box
- Result Visualization** (plot_results.py) - Blue box
- Final Output: Algorithm Comparison Plots** (PNG graphs per workload & strategy) - Yellow box

Right Flow (AWS EC2 Instance):

- AWS EC2 Instance** (upload code + docker) - Green box
- Build Docker image** / **Run hybrid-api (8080)** - Blue box
- Prometheus Container Scrape / metrics** - Blue box
- Grafana Visualization Dashboard (P50/P90...)** - Blue box
- EC2 Terminal Testing** curl /submit requests - Orange box

1. Local Simulation

2. Cloud Experiment (AWS EC2)

This setup allows for the evaluation of the scheduler's performance under real cloud loads.

Average Wait Time per Strategy (peak)

Strategy	Average Wait Time (peak)
edf	0
hrm	60
hybrid	0
rr	0
sjf	60

Average Turnaround Time per Strategy (peak)

Strategy	Average Turnaround Time (peak)
edf	0
hrm	60
hybrid	0
rr	0
sjf	60

Average Wait Time per Strategy (mixed)

Strategy	Average Wait Time (mixed)
edf	0
hrm	~100
hybrid	~100
rr	8000
sjf	~100

Average Turnaround Time per Strategy (mixed)

Strategy	Average Turnaround Time (mixed)
edf	0
hrm	~100
hybrid	~100
rr	8000
sjf	~100

Average Wait Time per Strategy (batch)

Strategy	Average Wait Time (batch)
edf	0
hrm	~100
hybrid	~100
rr	17500
sjf	~100

Average Turnaround Time per Strategy (batch)

Strategy	Average Turnaround Time (batch)
edf	0
hrm	~100
hybrid	~100
rr	17500
sjf	~100

Local Simulation



This project successfully demonstrated a hybrid scheduling system that can run stably in both local and cloud environments. It achieves real-time monitoring of task performance, latency, and throughput by containerizing the scheduler and integrating Prometheus and Grafana. The results show that this hybrid scheduling system improves task allocation efficiency. In summary, this project validates the feasibility of the scheduling system in both local and cloud environments, providing a practical foundation for further optimization of scheduling strategies.